CS3230 Lecture 07-08

Introduction to NP-completeness

Lecture Topics and Readings:

- Complexity classes [CLRS] Ch34.1-34.2
- Polynomial-time reduction [CLRS] Ch34.3
- Circuit Satisfiability is NP-complete [CLRS] Ch34.3/Lecture 08
- More NP-complete problems [CLRS] Ch34.5/Lecture 08

I cannot find an efficient algorithm,
but neither can all these famous people.

Stephen Cook

- Computational Complexity Theory
- Turing award 1982
- Formalize the notations of polynomial time reduction
- Prove the existence of the *first* NP-complete problem "Boolean Satisfiability problem (SAT)"

Richard Karp

- Theory of Algorithm
- Turing award 1985
- Current research interest: Computational Biology
- Reducibility Among Combinatorial Problems
  - He proved 21 problems to be NP-complete!
- Max-flow problem
- Maximum matching in bipartite graph

Time Complexity Revisit

Deconstructing your mindset

Time Complexity Revisit

Consider a problem with input size **n**,
a polynomial time algorithm for this problem has worst case time complexity of $\mathbf{O(n^c)}$ for a constant **c**

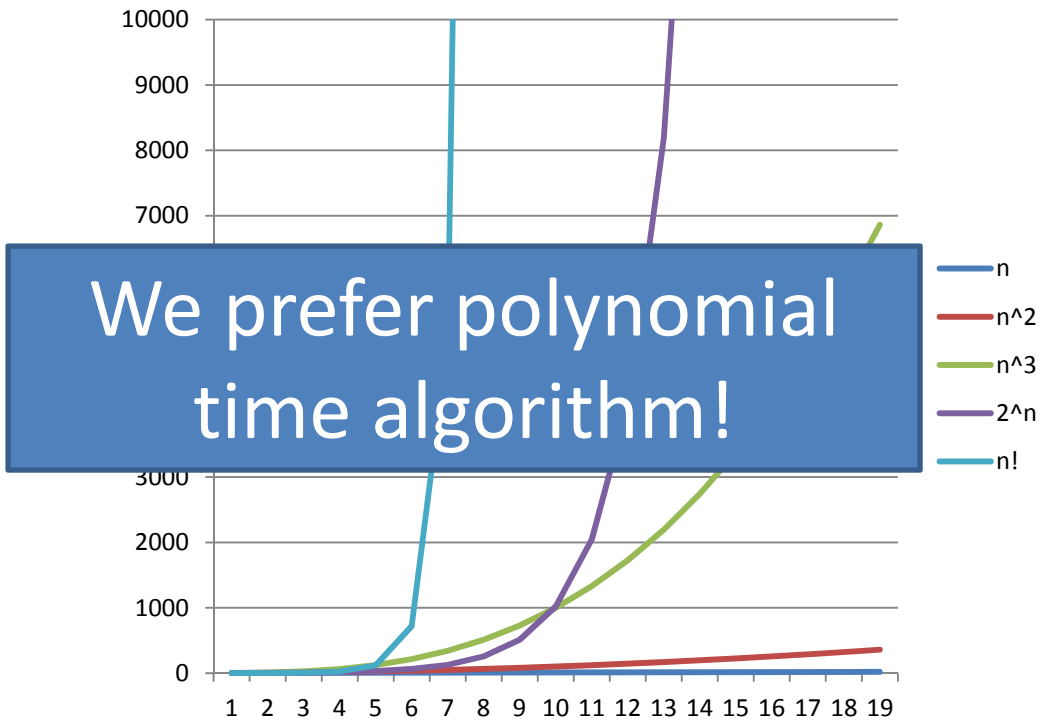
- For example: linear time $\mathbf{O(n)}$, quadratic time $\mathbf{O(n^2)}$, cubic time $\mathbf{O(n^3)}$, etc

On the other hand, a super-polynomial time algorithm has running time of $\boldsymbol{\omega(n^c)}$ for all constant **c**

- For example: exponential time $\mathbf{O(2^n)}$, factorial time $\mathbf{O(n!)}$, etc

Illustration of Time Complexities

When **n** increases, a super-polynomial time algorithm runs **very slow**



We prefer polynomial time algorithm!

n	n^2	n^3	2^n	n!
1	1	1	2	1
2	4	8	4	2
3	9	27	8	6
4	16	64	16	24
5	25	125	32	120
6	36	216	64	720
7	49	343	128	5040
8	64	512	256	40320
9	81	729	512	362880
10	100	1000	1024	3628800
11	121	1331	2048	39916800
12	144	1728	4096	:O
13	169	2197	8192	…
14	196	2744	16384	
15	225	3375	32768	
16	256	4096	65536	
17	289	4913	131072	
18	324	5832	262144	
19	361	6859	524288	

Subset-SUM problem

Input: A set of integers **S** and an integer **t**

Output: Decide if there is a non-empty subset **S'** $\subseteq$ **S** such that $\sum_{x \in S'} x = t$

Examples:

- S = {-3, 9, 20, 21, 37, -43}, t = 0
  - Answer: Yes (Reason: -3+9+37-43 = 0)
- S = {-1, -3, -5, 10, 14, 20}, t = 0
  - Answer: No (Reason: no non-empty subset can have sum = 0)

A simple solution for Subset-SUM

Brute-force (a.k.a. "try-all") algorithm **SubsetSum(S, t)**

- For every subset $S' \subseteq S$, if $\sum_{x \in S'} x = t$, return YES;
- Return NO;

Running time analysis:

- Suppose $|S|=n$
- There are 2^n possible subsets S' of S
- It takes **O(n)** time to compute $\sum_{x \in S'} x$
- Hence, the running time is $\mathbf{O(n2^n)}$

Is the solution good?

The previous solution runs in exponential time :O

n	10	20	30	40	…	1000
$n2^n$ (seconds)	$\sim 10^4$	2×10^7	3×10^{10}	4×10^{13}	…	10^{304}

It is “impossible” to solve a problem of size 1000

Your boss asked, “The brute-force solution is too slow. Can you find a better solution?”

Thinking

You try to think of a fast solution…

However, you fail to find a more efficient method ☹

What should you do?

Try to show that Subset-SUM problem cannot be solved in polynomial time…

However, you fail to make a proof ☹

In fact, many famous people have tried,

but none of them can give a correct proof

How can you convince your boss that, for this problem, it is <u>unlikely</u> to have a polynomial time solution?

Possible response

You tell your boss, **“Subset-SUM problem is just as hard as many other problems that are widely recognized as difficult and no experts can solve them.”**

I can’t find an efficient algorithm,
but neither can all these famous people

[Garey & Johnson. Computers and Intractability: A Guide to the Theory of NP-Completeness. Freeman, 1979.]

Agenda

Lecture 07+08 discuss techniques that convince you that a problem is hard

Today, Week 07:

- Decision versus Optimization problems
- Complexity classes: P, NP, co-NP
- Polynomial-time reduction and NP-complete

Next Wednesday, Week 08:

- Circuit Satisfiability is NP-complete
- More NP-complete problems (as time permits)

Decision versus Optimization problems

Two sides of a coin

Decision vs. Optimization problems

Decision problem: Its solution is either YES or NO

e.g. The subset-sum problem is a decision problem

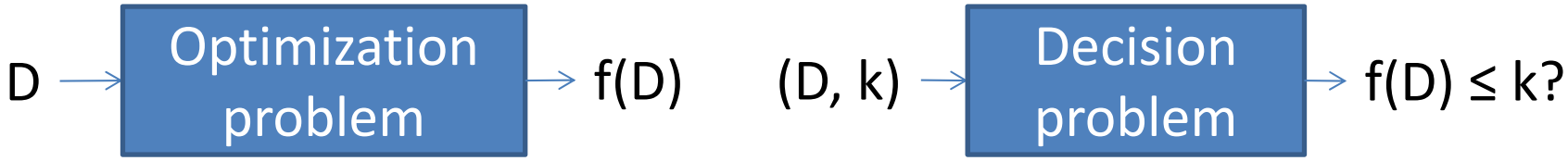
Optimization problem: Its solution is an optimal solution

e.g. The shortest path problem: Given a graph G, two vertices u and v, report a path from u to v with the fewest # of edges

Converting an optimization problem into a decision problem

Given an optimization problem with input **D** and we aim to optimize **f(D)**, we convert it into a decision problem by imposing a bound **k** on the value to be optimized

- e.g. The PATH problem (the decision problem of the shortest path problem): Given a graph **G**, 2 vertices **u** and **v**, a threshold **k**, decide if ∃ a path from **u** to **v** with at most **k** edges



Decision problem is easier (no harder) than optimization problem

Decision_Algorithm(D, k)

- Compute **f(D)** using **Optimization_Algorithm(D)**;
- If **f(D) ≤ k** then report "YES" else report "NO";

If we know how to solve the optimization problem, we know how to solve the decision problem, thus:

- Decision problem is <u>easier (no harder)</u> than optimization problem

In Lecture 07+08, we only study **decision** problems

Complexity classes: P, NP, co-NP

Leaving the world of <u>only P</u> problems

Several classes of problems

- Class P:
  - The set of problems that **can be solved in polynomial time**, i.e., $O(n^c)$ where c is a constant
- Class NP:
  - The set of (decision) problems that are "verifiable" in polynomial time when solution **exists** (the answer is a YES)
- Class co-NP:
  - The set of (decision) problems that are "verifiable" in polynomial time when solution **does not exist** (the answer is a NO)

P (Polynomial time problems)

The problems in **P** can be solved in **polynomial time**

Examples:

- Sorting
- (Single-Source/All-Pairs) Shortest Path
- Minimum Spanning Tree
- Matrix Multiplication
- And basically almost all algorithms that you have seen in CS1010, CS1020, and CS2010 (or CS2020) so far…

NP problems

The problems in this class **NP** can be "verifiable" to have a "YES" answer in polynomial time

- Suppose the problem has a solution
- Given a **certificate** of the solution, we can verify the correctness of the solution in polynomial time

Example: Subset-SUM from earlier

- Certificate: The subset S'
- The verification algorithm check if $\sum_{x \in S'} x = t$
- Time: $O(|S|) = O(n)$
- Hence, the Subset-SUM problem is in NP

What does NP stand for?

A. No problem

B. Non-polynomial time problem

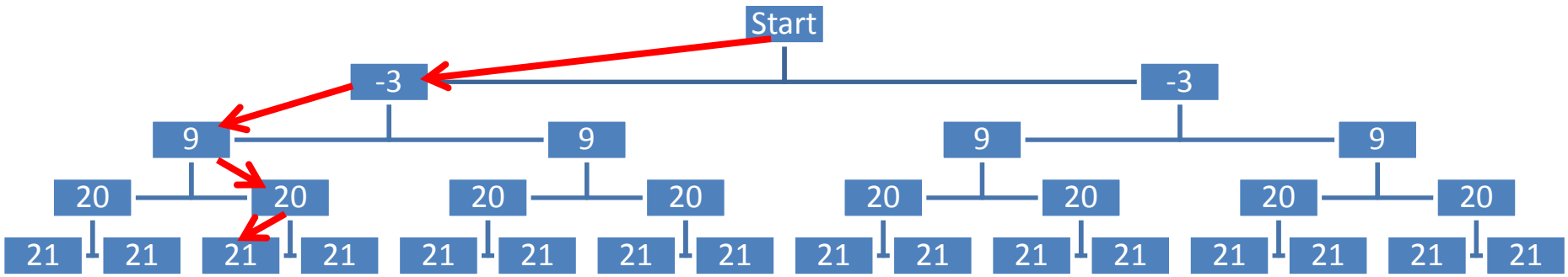
C. Non-deterministic polynomial time problem

Subset-SUM problem (revisited)

Example: S = {-3, 9, 20, 21}, t = 27

A **non-deterministic** machine works as follows:

(left sub-tree: take, right sub-tree: do not take)



A **deterministic machine** (computer today) can't do that

co-NP

A problem **X** $\in$ **co-NP** if it takes polynomial time to verify if the result is "<u>NO</u>" given a **certificate**

For example, **tautology** (next slides) is in co-NP

Definition: A Boolean formula

A Boolean formula F(X) formed by:

1. a set of Boolean variables $X = \{x_1, \ldots, x_k\}$,
2. negations ~,
3. and $\wedge$,
4. or $\vee$,
5. parentheses.

Example: $F(\{x_1, x_2, x_3\}) = (x_1 \vee \neg x_2) \wedge (x_2 \vee \neg x_3)$

- If $x_1=1$, $x_2=0$, $x_3=1$, then $F(X) = 0$
- If $x_1=1$, $x_2=0$, $x_3=0$, then $F(X) = 1$

Tautology

Input: A Boolean function F(X)

Output: "YES" if F(X) evaluates to 1 for <u>all assignments</u> to the Boolean variables, "NO" otherwise; Examples:

- $F(X) = (x_1 \vee \neg x_1) \wedge (x_2 \vee \neg x_2)$
  - Answer: "YES"
- $F(X) = (x_1 \vee \neg x_2) \wedge (x_2 \vee \neg x_1)$
  - Answer: "NO" (Reason: when $x_1 = 0$, $x_2 = 1$, $F(X) = 0$)

Certificate: The assignment X s.t. F(X) <u>evaluates to 0</u>

- The verification algorithm verifies F(X) = 0 in polynomial time
- This is a "NO", hence, tautology is in co-NP

Is there any problem which is in both NP and co-NP?

- Yes!

$P \subseteq NP$ and $P \subseteq$ co-NP

Proof: If a problem **X** is in **P**, let **Y** be your polynomial time algorithm for the problem **X**

- You just ignore the certificate
- By running your program **Y** in polynomial time, then you can decide if the answer is 'YES' or 'NO'

Hence, $P \subseteq (NP \cap \text{co-NP})$

Is there any problem which is **not in** P but is **in** both NP and co-NP?

- May be!

Integer factorization

Input: Two positive integers **m** and **t** with **t < m**

Output: "YES" if **m** is divisible by **f** where **1 < f ≤ t**; otherwise, "NO"; Example:

- When (m=3741, t=5), output "YES" as 3741 is divisible by 3
- When (m=17017, t=6), output "NO" as 17017 = 7*11*13*17

We still <u>cannot</u> find a polynomial time algorithm (in the length of **m**) for integer factorization

- Public-key cryptography relies on this fact!
- We will see that integer factorization is in both NP and co-NP

Integer factorization is in NP

Certificate: A factor **f** that divides **m**

In polynomial time, we can check if:

1. $1 < \mathbf{f} \leq \mathbf{t}$, and
2. Integer **m** is divisible by **f**

Example:

- When (m=3741, t=5),
  output "YES" since 3741 is divisible by 3

Integer factorization is in co-NP

Certificate: The prime factorization of **m**, i.e., $\mathbf{m = f_1 f_2 \ldots f_k}$

In polynomial time, we can verify that

1. Integer $\mathbf{m} = \mathbf{f_1 f_2 \ldots f_k}$
2. none of the factors is smaller than **t**
3. Using AKS primality test* (2002), we can verify $f_1, f_2, \ldots, f_k$ are prime numbers in polynomial time

   Ref: Manindra **A**grawal, Neeraj **K**ayal, Nitin **S**axena, "PRIMES is in P", *Annals of Mathematics* 160 (2004), no. 2, pp. 781-793

Example:

- When (m=17017, t=6), output "NO" since 17017 = 7*11*13*17

Relationship between P, NP and co-NP

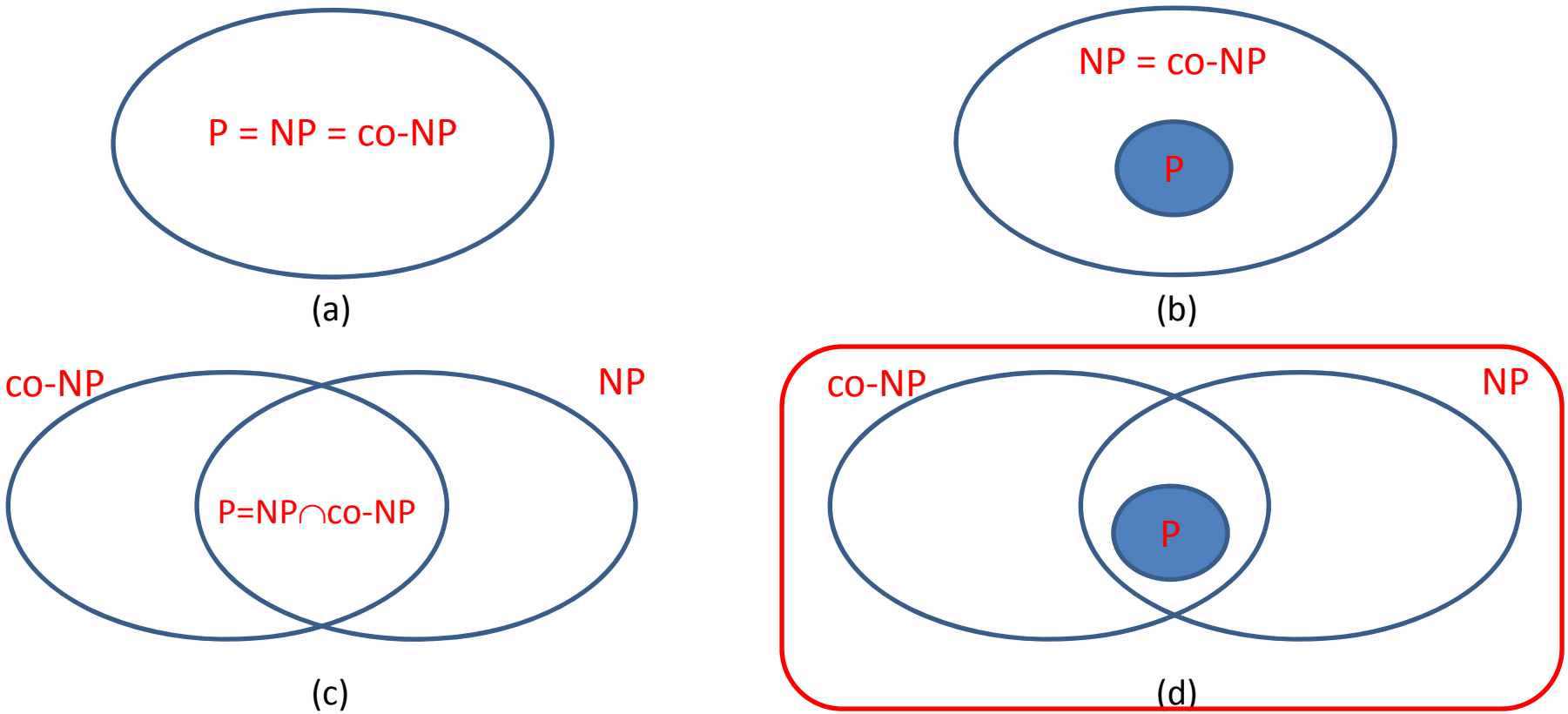


Figure 34.3 Four possibilities for relationships among complexity classes. In each diagram, one region enclosing another indicates a proper-subset relation. **(a)** P = NP = co-NP. Most researchers regard this possibility as the most unlikely. **(b)** If NP is closed under complement, then NP = co-NP, but it need not be the case that P = NP. **(c)** P = NP ∩ co-NP, but NP is not closed under complement. **(d)** NP ≠ co-NP and P ≠ NP ∩ co-NP. Most researchers regard this possibility as the most likely.

P = NP?

A millennium prize problem that has been open since 1971 and one of the most important problems in mathematics and theoretical computer science

People *generally think* that **P≠NP**

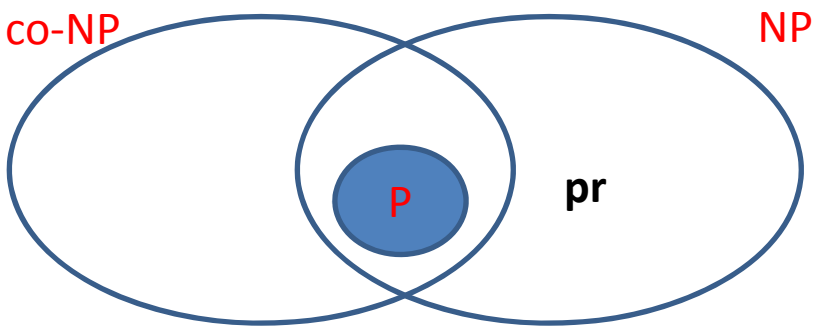
If **P=NP**, it will <u>greatly affect</u> the practical world

- Eg1: It will break many cryptographic solutions, including public-key cryptography
- Eg2: Protein structure prediction (which is in NP-complete) becomes feasible
- Eg3: Many operational research problems, including integer programming, becomes solvable

Theorem: If NP ≠ co-NP, then P ≠ NP

Proof:

- If **NP ≠ co-NP**, there exists some problem **pr** in **NP** and not in **co-NP**
- Recall that **P** $\subseteq$ **(NP∩co-NP)**
- Hence, **pr** is not in **P**
- Thus, **P ≠ NP**



Term test

- 7 Mar 2015
- 10:00am – 11:30am
- LT15

- Open book
- Cover all topics up to today lecture

HW3 Deadline Reminder

HW2 is still about the first half of CS3230

- Due date: Thursday, 12-Mar-2015, before 12:00noon
- Submit to the DropBox outside Prof Leong's office

Week 8 is E-learning week

- Lecture 8
  - Webcast only!

- Tutorial: We have 4 sessions for you to ask questions.
  - Prof. Leong: Mon 2-3pm
  - Prof. Ken Sung: Mon 3-4pm
  - Oh Shunhao: Mon 5-6pm
  - Nguyen Nam Ninh: Fri 2-3pm

- You can also talk to you tutor through discussion forum

Polynomial Time Reduction

A very important technique!

Definition: Polynomial time reduction

Consider two problems **A** and **B**

A can be solved as follows:

- Input: An instance α of **A**
  1. Convert α to an instance β of **B**
  2. Solve β and obtain a solution
  3. Based on the solution of β, we obtain the solution of α

If steps 1 and 3 take <u>polynomial time</u>, we say $\mathbf{A} \leq_p \mathbf{B}$

Example: Matrix Multiplication/Square

Mat_multi

(Matrix Multiplication)

Input:
Two **n x n** matrices **A** and **B**

Output:
A x B

Mat_sqr

(Matrix Square)

Input:
One **n x n** matrix **C**

Output:
$\mathbf{C^2}$

Mat_sqr $\leq_p$ Mat_multi

Recall: A $\leq_{\mathbf{p}}$ B means that problem A is reduced to problem B in **p**olynomial time

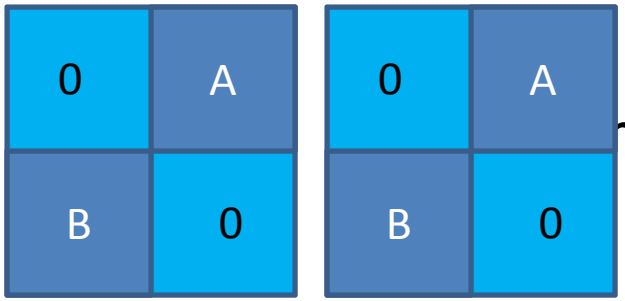
Mat_sqr $\leq_p$ Mat_multi reduction is trivial

Reason: We can assign **A=C** and **B=C**, run Mat_multi, and then, we obtain $\mathbf{C^2}$

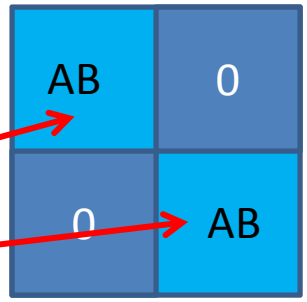
Mat_multi $\leq_p$ Mat_sqr?

Input: **A=**

and **B=**

Transform them to **C=** 

mial time

After we call Mat_sqr, we obtain $\boldsymbol{C^2}$**=** 

Notice that **AB** is there, hence, Mat_multi $\leq_p$ Mat_sqr

(Polynomial-time) reduction between decision problems

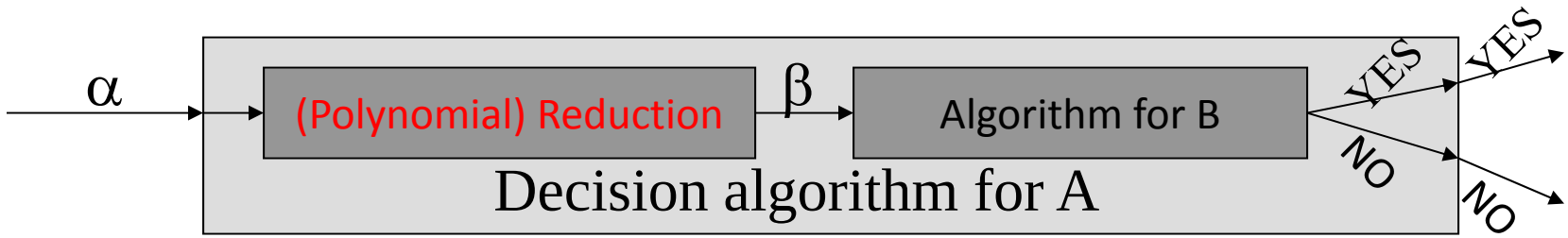
Consider two decision problems A and B

- Let α be an instance of the decision problem A and β be an instance of the decision problem B

A reduction from A to B ($A \leq_p B$) is a transformation with the following properties:

- The transformation takes polynomial time
- The answer is the same (the answer for α is YES if and only if the answer for β is YES, and vice versa)

Suppose $A \leq_p B$



Implications:

1. If B can be solved in polynomial time, so does A
2. If A is hard, so does B

NP-complete (NPC)

A problem **L** $\in$ **NP-hard** if <u>any other</u> problem **L** $\in$ **NP** can be reduced to **L** in polynomial time (i.e. **L'** $\leq_p$ **L**)

So if **L** can be solved in polynomial time, then <u>**all problems in NP**</u> can be solved in polynomial time :O

If **L** $\in$ **NP-hard** <u>and</u> **L** $\in$ **NP**,
then **L** $\in$ **NP-complete (NPC)**

Review: Shortest vs Longest Path

Consider an unweighted directed graph $G = (V, E)$

- Let $n = |V|+|E|$ be the input size

The **shortest path** problem asks for a shortest (always simple) path but the **longest path** problem asks for a longest simple path (of length k or more)

Both problems are in **NP**

- The shortest path problem can be solved in $O((|V|+|E|) \log |V|) = O(n^2)$ time; Hence, it is in **P**
- On the other hand, the longest path problem is in **NPC** (proof deferred until tutorial)

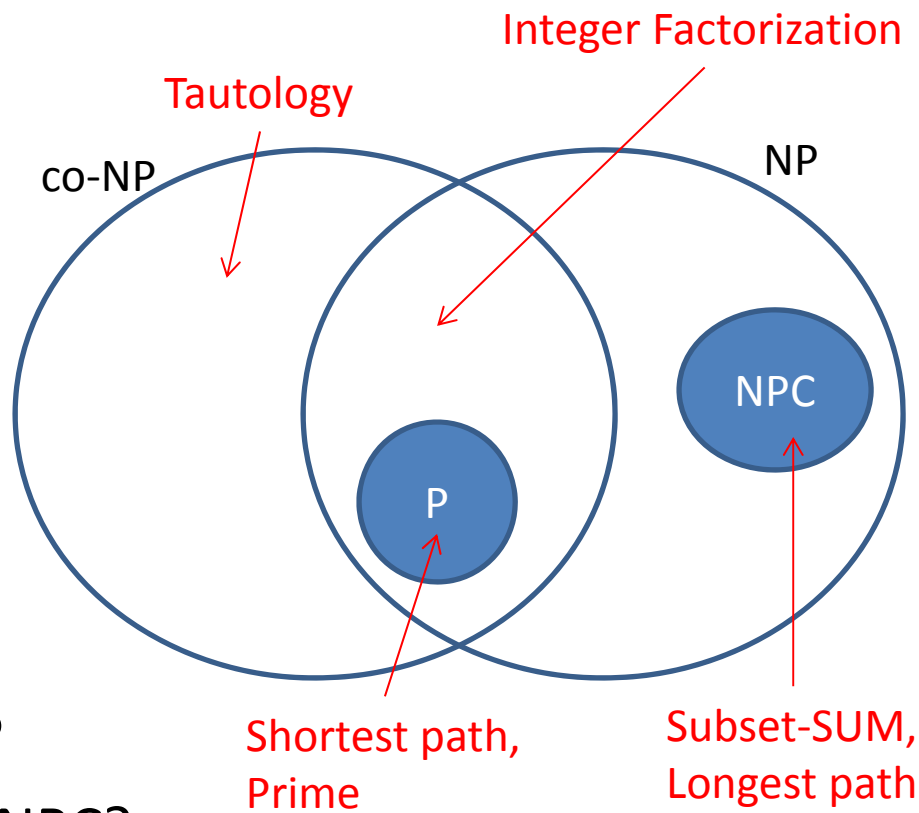
Relationship among P, NP, co-NP, NPC

Known relationships:

- $P \subseteq (NP \cap co\text{-}NP)$
- $NPC \subseteq NP$

Open problems:

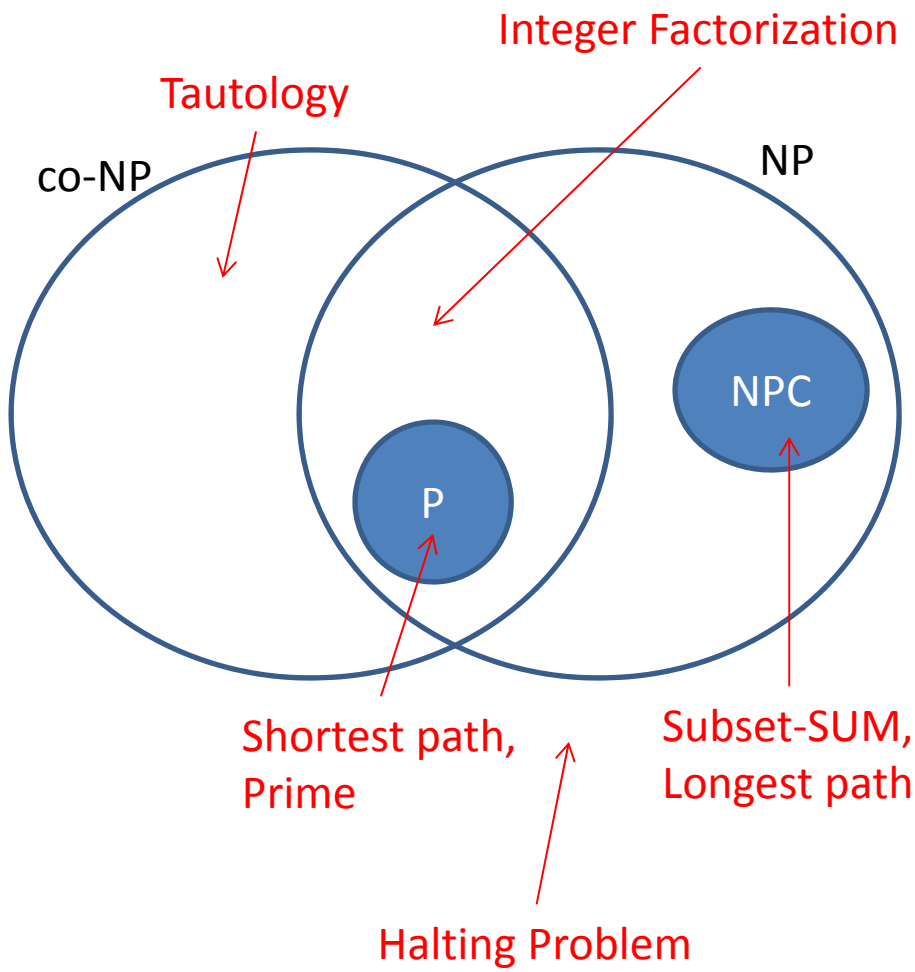
- Is P = NP?
- Is P a proper subset of NP?
- Is NPC a proper subset of NP?
- Is there any problem in NP-P-NPC?



The Halting Problem

- INPUT: A string P and a string S.
  - We will think of P as a program.
  - We will think of S as the input of the program P.
- OUTPUT:
  - 1 if P halts on S, and
  - 0 if P goes into an infinite loop on S.

- This problem is undecidable. It is not solvable by computer. (We skip the proof!)



Question: Halting problem is

- (a) P
- (b) NP
- (d) co-NP
- (d) NP-complete
- (e) NP-hard ✓ Exercise!
- (d) none of the above

Why do we want to determine if a problem is in NPC?

If a problem is in NPC, it is a good evidence that the problem is intractable (i.e. hard)

Instead of wasting time to find a good solution, we will try other strategies, e.g.

- Solve the **special case(s)** of the problem optimally (Week 09)
- Use efficient **Dynamic Programming or even "Brute Force"** to solve *reasonable size* of the problem (Week 10)
- Use **greedy algorithm** that will work for *some cases* (Week 11)
- Use good **approximation algorithm** to the problem (Week 12)

Next Lecture 08

1. Proof of the 1st NPC problem
   (to kick start the other proofs)
2. Reducing (several) other hard problems to known NPC problems (as time permits)